

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO5 – Articulation	Assessment:	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	N.R.Yasaswini	Reg. No.:	192425119
Section / Batch:	AI&ML	Date:	26-08-2026

Code of Conduct

I _N.R.Yasaswini(192425119)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ___NRY_____

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Reference Resolution Using Multiple Constraints

Design an algorithm to perform **Reference Resolution** by identifying pronouns and selecting the most appropriate antecedent using gender, number, recency, and semantic compatibility.

Apply your algorithm to the discourse:

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Generate possible antecedents for each pronoun.
3. Apply gender, number, recency, and semantic constraints.
4. Select the most appropriate antecedent.
5. Generate the resolved discourse.

Finally, explain how your algorithm avoids incorrect resolution when multiple antecedents have similar grammatical properties.

Algorithm:

Algorithm ReferenceResolution(Text)

1. Extract all entities and referring expressions.
2. Identify pronouns in each sentence.
3. For each pronoun, generate candidate antecedents from previous context.
4. Filter candidates using:
 - a. Gender agreement
 - b. Number agreement
 - c. Recency
 - d. Semantic compatibility
5. Assign a score to each valid candidate.
6. Select the candidate with the highest score.
7. Replace the pronoun with the selected antecedent.
8. Return the resolved discourse.

1. Entities: Meena, Kavya, laptop, project

Referring expressions: she, one, She, her, it

2–4. Resolution:

Pronoun	Candidates	Constraints	Resolution
she	Meena, Kavya	Female + singular + recency + context	Meena/Kavya*
one	laptop	Semantic compatibility	laptop
She	Meena, Kavya	Female + singular + recency	Meena*
her	Meena, Kavya	Female + singular + context	Kavya*
it	laptop	Object + semantic compatibility	laptop

The sentence is ambiguous because both Meena and Kavya are female and singular. A robust system therefore uses semantic/contextual scoring rather than gender alone. A plausible interpretation is that Kavya needed the laptop for her project, and Meena thanked Kavya and started using the laptop.

Coreference chain:

Meena ————— She ————— her
Kavya ————— she ————— her

Laptop ————— one ————— it

Evaluation: Gender and number eliminate incompatible candidates, while recency and semantic compatibility resolve many ties. When grammatical properties are identical, contextual/semantic scoring prevents relying only on recency.

2. Algorithm for Entity Chains and Discourse Coherence

Design an algorithm to identify **entity chains** in a discourse and use them to evaluate text coherence.

Apply your algorithm to the following text:

"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Link expressions referring to the same entity.
3. Construct entity chains.
4. Calculate or estimate the continuity of entities across sentences.
5. Determine whether the discourse is coherent.

Finally, explain how breaking an entity chain could reduce discourse coherence.

Algorithm:

Algorithm EntityChainCoherence(Text)

1. Split text into sentences.
2. Extract entities and referring expressions.
3. Compare expressions using coreference and semantic similarity.
4. Link expressions referring to the same entity.
5. Construct entity chains.
6. Count occurrences and continuity across sentences.
7. If important entities remain connected across sentences, classify the discourse as coherent.
8. Return chains and coherence score.

1. Entities: Anjali, laptop, processor, Python, computer, machine learning application.

2–3. Entity chains:

Anjali → She → Anjali

Laptop → The laptop → it → the computer

Python → Python

Processor → processor

Application → machine learning application

4. Continuity:

Sentence	Main entities
S1	Anjali, Laptop
S2	Laptop, Processor
S3	Anjali, Python, Laptop
S4	Anjali, Computer, Application

The **laptop/computer chain** and **Anjali chain** continue across multiple sentences.

5. Coherence: High / Coherent

Evaluation: The repeated references to *Anjali* and the *laptop/computer* connect the sentences logically.

If the entity chain were broken, for example, by replacing “She installed Python on it” with an unrelated

sentence, the connection between the actions and the laptop would weaken, reducing discourse coherence.

3. Algorithm for Identifying Discourse Relations

Design an algorithm to identify **discourse relations** between consecutive sentences using discourse markers and contextual meaning.

Apply your algorithm to:

"The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

Your algorithm should identify relations such as:

- Cause–Effect
- Sequence
- Problem–Solution
- Elaboration

Perform the following steps:

1. Divide the discourse into individual discourse units.
2. Identify explicit discourse markers.
3. Analyze the semantic relationship between units.
4. Assign an appropriate discourse relation.
5. Construct a discourse structure representation.

Finally, justify how the identified relations make the discourse logically coherent.

Algorithm:

Algorithm DiscourseRelation(Text)

1. Divide text into discourse units/sentences.
2. Detect discourse markers such as "as a result", "after that".
3. Analyze cause, effect, sequence and problem-solution relationships.
4. Assign the strongest discourse relation.
5. Construct a discourse tree.
6. Return the relations and coherence explanation.

Discourse Units:

- D1: The server crashed unexpectedly.
- D2: Users could not access the application.
- D3: The technical team restarted the server.
- D4: The system became available again.

Relations:

D1 —Cause—► D2

Server crash → Users cannot access application

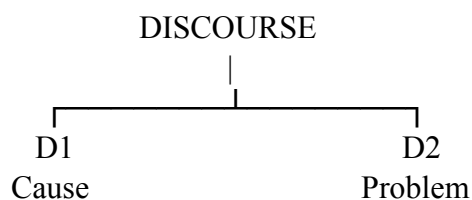
D2 —Problem—► D3

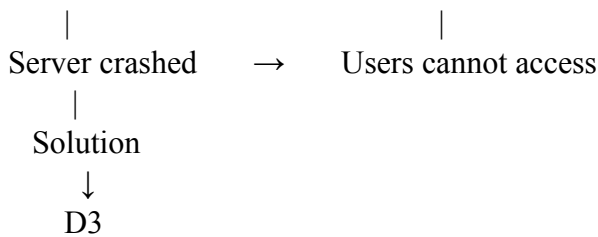
Access failure → Team restarts server

D3 —Sequence—► D4

Restart → System becomes available

Discourse structure:





indicates **Cause–Effect**, while “After that” indicates **Sequence**. The restart acts as a **Solution** to the access problem. Therefore, the discourse has a clear logical flow.

4. Algorithm for Dialogue Act Classification

Design an algorithm to recognize **Dialogue Acts** in a conversation.

Apply your algorithm to:

User: "Hello, I need help with my assignment."

Agent: "Sure, what problem are you facing?"

User: "I do not understand machine learning."

Agent: "I can explain the basic concepts to you."

Classify each utterance into one of the following:

- Greeting
- Request
- Question
- Inform
- Offer
- Confirmation

Perform the following steps:

1. Preprocess each utterance.
2. Extract relevant linguistic features.
3. Identify the communicative intention.
4. Assign a dialogue act.
5. Generate the dialogue-act sequence.

Finally, explain how dialogue-act recognition helps a conversational agent select an appropriate next response.

Algorithm:

Algorithm DialogueActClassification(Conversation)

1. Preprocess each utterance.
2. Extract keywords, sentence type and speech intention.
3. Detect greeting, request, question, inform, offer or confirmation.
4. Assign the appropriate dialogue act.
5. Generate the dialogue-act sequence.
6. Use the act to select the next response.

Classification:

Speaker	Utterance	Dialogue Act
User	“Hello, I need help with my assignment.”	Greeting + Request
Agent	“Sure, what problem are you facing?”	Question
User	“I do not understand machine learning.”	Inform

Speaker	Utterance	Dialogue Act
Agent	"I can explain the basic concepts to you."	Offer

Dialogue-act sequence:

Greeting/Request → Question → Inform → Offer

Evaluation: Dialogue-act recognition identifies the user's intention. For example, after an **Inform** act, the agent can provide an explanation, while after a **Question**, it should provide an answer.

5. Algorithm for Dialogue State Tracking

Design an algorithm for a **Conversational Agent** that maintains dialogue state while collecting information from a user.

Apply your algorithm to:

User: "I want to book a flight."

Agent: "Where would you like to travel?"

User: "I want to go to Delhi."

Agent: "From which city?"

User: "From Chennai."

Your algorithm should maintain the following slots:

Departure City

Destination City

Travel Date

Number of Passengers

Perform the following steps:

1. Initialize the dialogue state.
2. Identify information provided in each utterance.
3. Update the corresponding slot.
4. Identify missing information.
5. Generate the next appropriate question.

Finally, explain how dialogue state tracking improves coherence in a multi-turn conversation.

Algorithm:

Algorithm DialogueStateTracking(Conversation)

1. Initialize all slots as NULL.
2. Process each user utterance.
3. Extract entities and values from the utterance.
4. Match each value with its corresponding slot.
5. Update the dialogue state.
6. Find missing required slots.
7. Ask the next question for a missing slot.
8. Repeat until all required information is collected.

Initial state:

Departure City = NULL

Destination City = NULL

Travel Date = NULL

Number of Passengers = NULL

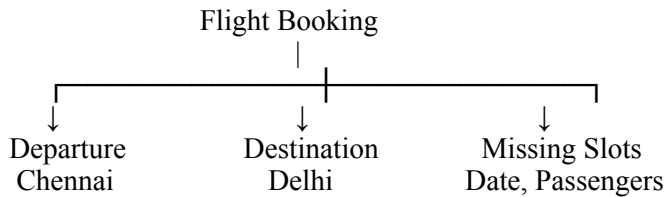
State updates:

Utterance	State Update
"I want to book a flight."	No slot value; booking intent detected

Utterance	State Update
“I want to go to Delhi.”	Destination = Delhi
“From Chennai.”	Departure = Chennai

Current dialogue state:

Departure City = Chennai
Destination City = Delhi
Travel Date = NULL
Number of Passengers = NULL
State representation:



Evaluation: Dialogue State Tracking preserves information from previous turns, prevents the agent from repeatedly asking for known details, and maintains a coherent multi-turn conversation until all required slots are filled.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %

1	4	4	4	3	3	18	97
2	4	4	4	4	4	20	
3	4	4	4	4	4	20	
4	4	4	4	4	4	20	
5	4	4	4	4	3	19	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____